

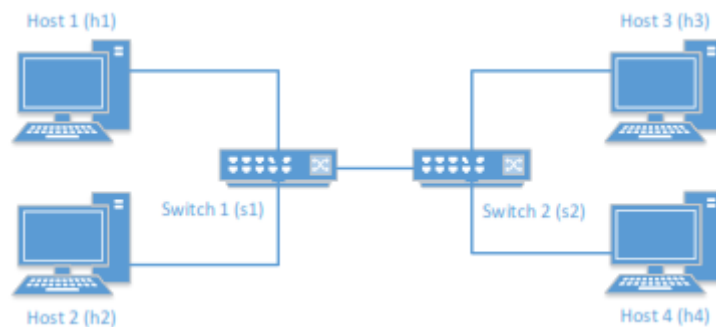
Custom Topologies:

Objective: In this lab, we will look at writing our code to create SDN topologies.

Introduction:

Mininet supports a simple Python API to create custom network topologies. You can make your custom topology by writing a few lines of Python code. The figure below shows a topology with two switches and four hosts. The code consists of a class named `MyFirstTopology` which extends the `Topo` class provided by the API.

You can use any text editor GUI or text-based. Save the source code in a file named `myfirsttopology.py` in your mininet directory.



The source code for the custom topology is shown in the above figure.

```
from mininet.topo import Topo

class MyFirstTopology( Topo ):
    "Simple topology example."

    def build( self ):
        "Create custom topo."

        # Add hosts and switches
        h1 = self.addHost( 'h1' )
        h2 = self.addHost( 'h2' )
        h3 = self.addHost( 'h3' )
        h4 = self.addHost( 'h4' )
        leftSwitch = self.addSwitch( 's1' )
        rightSwitch = self.addSwitch( 's2' )

        # Add links
        self.addLink( h1, leftSwitch )
        self.addLink( h2, leftSwitch )
        self.addLink( leftSwitch, rightSwitch )
        self.addLink( rightSwitch, h3 )
        self.addLink( rightSwitch, h4 )

topos = { 'myfirsttopology': ( lambda: MyFirstTopology() ) }
```

Type the following command.

Sudo mn --custom myfirsttopology.py --topo myfirsttopology

```
root@saimvm:/home/saim/mininet/mininet/custom# ls
myfirsttopology.py  README  topo-2sw-2host.py
root@saimvm:/home/saim/mininet/mininet/custom# sudo mn --custom myfirsttopology
.py --topo myfirsttopology
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2
*** Adding links:
(h1, s1) (h2, s1) (s1, s2) (s2, h3) (s2, h4)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 2 switches
s1 s2 ...
*** Starting CLI:
mininet>
```

Now test your topology by using the ping command. For example, H1 ping h2, h1 ping s2 etc.

If you want to run a quick test without entering the CLI, you can use options --test in Mininet, as follows

sudo mn --custom myfirsttopology.py --topo myfirsttopology --test pingall

```
root@saimvm:/home/saim/mininet/mininet/custom# sudo mn --custom myfirsttopology
.py --topo myfirsttopology --test pingall
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2
*** Adding links:
(h1, s1) (h2, s1) (s1, s2) (s2, h3) (s2, h4)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 2 switches
s1 s2 ...
*** Waiting for switches to connect
s1 s2
```

```
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
*** Stopping 1 controllers
c0
*** Stopping 5 links
.....
*** Stopping 2 switches
s1 s2
*** Stopping 4 hosts
h1 h2 h3 h4
*** Done
completed in 5.536 seconds
root@saimvm:/home/saim/mininet/mininet/custom#
```

TASK:

Now assume that we want to accomplish the following:

1. Create the topology shown in the above figure
2. Start the network
3. Show the detailed connectivity of all nodes in the network
4. Test the connectivity by pinging all nodes

We can automate this sequence of operations by writing a Python script. This helps us to automate network experiments. To this end, we extend the previous source code, as shown in the above figure.

```

1. #!/usr/bin/python
2.
3. from mininet.topo import Topo
4. from mininet.net import Mininet
5. from mininet.util import dumpNodeConnections
6. from mininet.log import setLogLevel
7.
8. class MyFirstTopology( Topo ):
9.     "Simple topology example."
10.    def __init__( self ):
11.        "Create custom topo."
12.        # Initialize topology
13.        Topo.__init__( self )
14.        # Add hosts and switches
15.        h1 = self.addHost( 'h1' )
16.        h2 = self.addHost( 'h2' )
17.        h3 = self.addHost( 'h3' )
18.        h4 = self.addHost( 'h4' )
19.        leftSwitch = self.addSwitch( 's1' )
20.        rightSwitch = self.addSwitch( 's2' )
21.        # Add links
22.        self.addLink( h1, leftSwitch )
23.        self.addLink( h2, leftSwitch )
24.        self.addLink( leftSwitch, rightSwitch )
25.        self.addLink( rightSwitch, h3 )
26.        self.addLink( rightSwitch, h4 )
27.
28.    def runExperiment():
29.        "Create and test a simple experiment"
30.        topo = MyFirstTopology( )
31.        net = Mininet(topo)
32.        net.start()
33.        print ( 'Dumping host connections' )
34.        dumpNodeConnections(net.hosts)
35.        print ( 'Testing network connectivity' )
36.        net.pingAll()
37.        net.stop()
38.
39.    if __name__ == '__main__':
40.        # Tell mininet to print useful information
41.        setLogLevel('info')
42.        runExperiment()

```

Save this file with the name of myfirstprogram.py and in the folder of custom or examples.

Use the following command to run this first program.

```
Python3 /path/to/file/myfirstprogram.py
```

```
root@saimvm:/home/saim/mininet# python3 /home/saim/mininet/mininet/custom/myfirstprogram.py
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2
*** Adding links:
(h1, s1) (h2, s1) (s1, s2) (s2, h3) (s2, h4)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 2 switches
s1 s2 ...
Dumping host connections
```

```
Dumping host connections
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s2-eth2
h4 h4-eth0:s2-eth3
Testing network connectivity
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
*** Stopping 1 controllers
c0
*** Stopping 5 links
.....
*** Stopping 2 switches
s1 s2
*** Stopping 4 hosts
h1 h2 h3 h4
*** Done
root@saimvm:/home/saim/mininet#
```

Remember to clean up (sudo mn -c).

Mininet Python Classes

The Mininet Python API consists of several Python classes, such as Topo, Mininet, Host, Switch, Link and their subclasses. The API is built at three primary levels which provide different levels of abstraction for customising the network topologies. Here is a brief description of the levels:

- Low-level API: This API includes the base node and link classes such as Host, Switch and Link.

You can directly instantiate some objects from these classes to create a network.

- Mid-level API: This API consists of the Mininet class which provides several methods, such as addHost(), addSwitch(), and addLink(), for creating a network.
- High-level API: This API consists of the Topo class, which provides the ability to create reusable, parameterised topology templates. These templates can be passed to the mn command (via the – custom option as in Part 2 above) and used from the command line.

The above two codes are the two examples that show how to use the high-level APIs which also provide an object-oriented framework by presenting the topo class. The code below shows the low-level API for creating custom topology. Observe the code below where objects are created, and topology is connected manually. The code can be tested by typing the following command.

```
sudo python myfirsttopology_lowlevel.py
```

```
1. #!/usr/bin/python
2.
3. from mininet.node import Host, OVSSwitch, Controller
4. from mininet.link import Link
5.
6. h1 = Host( 'h1' )
7. h2 = Host( 'h2' )
8. h3 = Host( 'h3' )
9. h4 = Host( 'h4' )
10. s1 = OVSSwitch( 's1', inNamespace=False )
11. s2 = OVSSwitch( 's2', inNamespace=False )
12. c0 = Controller( 'c0', inNamespace=False )
13. Link( h1, s1 )
14. Link( h2, s1 )
15. Link( h3, s2 )
16. Link( h4, s2 )
17. Link( s1, s2 )
18. h1.setIP( '10.0.0.1/24' )
19. h2.setIP( '10.0.0.2/24' )
20. h3.setIP( '10.0.0.3/24' )
21. h4.setIP( '10.0.0.4/24' )
22. c0.start()
23. s1.start( [ c0 ] )
24. s2.start( [ c0 ] )
25. print( h1.IP )
26. print( h2.IP )
27. print( h3.IP )
28. print( h4.IP )
29. print( 'Pinging ...' )
30. print( h1.cmd( 'ping -c3', h2.IP() ) )
31. print( h1.cmd( 'ping -c3', h3.IP() ) )
32. s1.stop()
33. s2.stop()
34. c0.stop()
```

```
*** Cleanup complete.
root@saimvm:/home/saim/mininet# sudo python3 /home/saim/mininet/mininet
lowlevel.py
<bound method Node.IP of <Host h1: h1-eth0:10.0.0.1 pid=23320> >
<bound method Node.IP of <Host h2: h2-eth0:10.0.0.2 pid=23322> >
<bound method Node.IP of <Host h3: h3-eth0:10.0.0.3 pid=23324> >
<bound method Node.IP of <Host h4: h4-eth0:10.0.0.4 pid=23326> >
Pinging....
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=21.7 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.238 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.061 ms

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2022ms
rtt min/avg/max/mdev = 0.061/7.331/21.695/10.156 ms

PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=3.59 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=1.13 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.093 ms

--- 10.0.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 0.093/1.602/3.587/1.465 ms

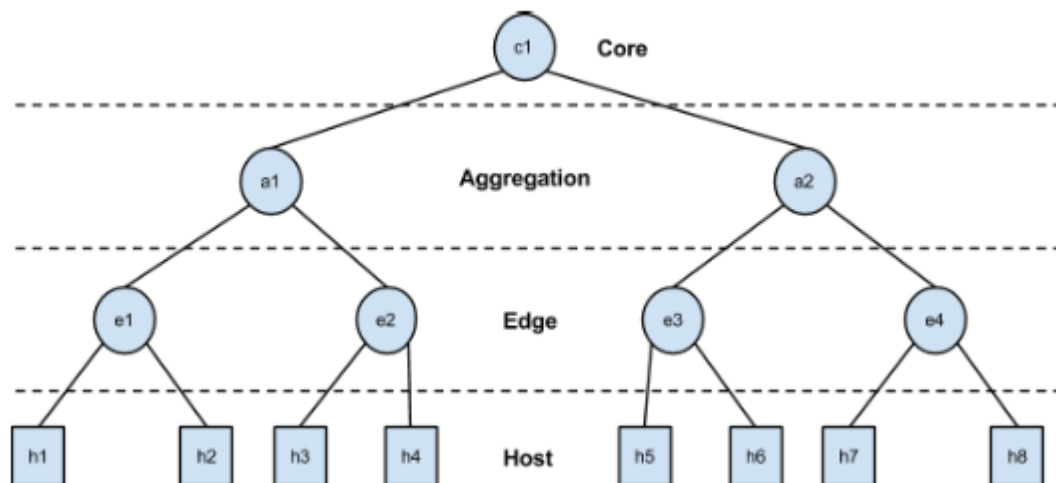
root@saimvm:/home/saim/mininet#
```



TASK 2:

** You need to save the solution to submit when asked.

Many data centres use a tree-like topology, as shown in the following figure. End hosts (Servers) connect to top-of-rack switches (edge switches), and form the leaves of the tree, one or more core switches form the root of the tree, and one or more aggregation switches form the intermediate nodes of the tree. In a simple tree topology, there is only one core switch connected to n aggregation switches, each aggregation switch is connected to n edge switches, and each edge switch is connected to n hosts (Servers). The following figure shows a simple tree topology where $n=2$.



Question 1: Write a Python program to use the high-level API and create a simple-tree data centre topology. You must follow the naming system as shown in the figure above. You must save your source code; it might be required at the end of the module. Name it as `simpletree_highlevel.py`. When finished show your instructor.

Question 2: From any of the above questions, check the connectivity and show using screenshots and write your observations. Now stop the core switch using the command `switch c1 stop`, check the connectivity again and report your observations.

Additional question for programming practice (Challenge question):

Write the program in a way that it asks for tree levels as n (e.g, $n=2$) and then create a tree topology for n levels.